

# Game Forge — System Performance & Evaluation Report

**Data sources:** live production containers `game-forge-mongo` (MongoDB 8.2.11) and `game-forge-minio` (MinIO 2025-09-07), extracted 2026-07-02.

**Observation window:** 2026-06-20 → 2026-07-01 (12 days of real usage).

## 1. Executive Summary

| Metric                                       | Value                                        |
|----------------------------------------------|----------------------------------------------|
| Generation pipeline runs recorded            | 33                                           |
| Runs reaching a terminal state               | 20 (13 completed, 7 failed)                  |
| <b>Pipeline success rate (terminal runs)</b> | <b>65 %</b>                                  |
| Mean end-to-end generation time (successful) | <b>276.5 s (≈ 4 min 37 s)</b>                |
| Median end-to-end generation time            | 268.2 s                                      |
| Fastest / slowest successful run             | 109.7 s / 453.1 s                            |
| Assets generated & stored                    | 154 asset records, 100 % generated           |
| Playable web builds produced                 | 13 (all target <code>web</code> , Godot 4.x) |
| Mean deliverable bundle size                 | 67.7 MB (6 files per build)                  |
| Total object storage consumed                | 2.2 GiB / 1,874 objects / 1 bucket           |
| Registered users / projects                  | 3 users / 36 projects                        |

The remaining 13 runs are stuck in `processing` — orphaned jobs whose containers were stopped mid-run; they never reached a terminal state and are excluded from success-rate and latency statistics.

## 2. Generation Pipeline Performance (MongoDB generationjobs , n = 33)

### 2.1 Outcome distribution

| Status                            | Count | Share  |
|-----------------------------------|-------|--------|
| completed                         | 13    | 39.4 % |
| processing (orphaned/interrupted) | 13    | 39.4 % |
| failed                            | 7     | 21.2 % |

### 2.2 Per-stage latency (completed stage executions, seconds)

| Stage                                     | n  | Mean  | Median (p50) | p90   | p95   | Min   | Max   | Std dev |
|-------------------------------------------|----|-------|--------------|-------|-------|-------|-------|---------|
| Planning (LLM game plan)                  | 29 | 42.4  | 33.0         | 82.5  | 85.3  | 10.5  | 158.6 | 31.7    |
| Assets (sprite/audio generation)          | 13 | 218.4 | 211.3        | 324.2 | 416.3 | 56.7  | 416.3 | 83.4    |
| Code (handoff checkpoint)                 | 13 | 0.006 | 0.006        | 0.007 | 0.007 | 0.006 | 0.007 | ~0      |
| Building (Godot headless export + upload) | 13 | 9.4   | 9.6          | 10.1  | 10.4  | 7.5   | 10.4  | 0.7     |

#### Observations

- **Asset generation dominates the pipeline:**  $\approx 79$  % of total wall-clock time on an average successful run (218 s of 277 s). Planning contributes  $\approx 15$  %, the Godot export  $\approx 3$  %.
- The **code stage records ~6 ms** because code artifacts are produced during the assets phase; the step acts as a pipeline checkpoint rather than a separate computation.
- The **build/export stage is highly deterministic** ( $\sigma = 0.72$  s), as expected for a fixed Godot headless export.
- Planning latency varies  $15\times$  (10.5 s  $\rightarrow$  158.6 s), reflecting LLM inference variance under provider load.

### 2.3 End-to-end latency (runs with a terminal state, seconds)

| Outcome   | n  | Mean  | Min   | Max   | Std dev |
|-----------|----|-------|-------|-------|---------|
| completed | 13 | 276.5 | 109.7 | 453.1 | 92.9    |
| failed    | 7  | 274.2 | 32.8  | 621.5 | 175.2   |

Median for completed runs: **268.2 s**; p90  $\approx$  402.4 s. Per-run data in [pipeline\\_runs.csv](#).

## 2.4 Failure analysis

All 7 hard failures occurred in the **assets stage**:

| Root cause (from job error logs)                                       | Occurrences |
|------------------------------------------------------------------------|-------------|
| Asset-service request aborted due to timeout                           | 3           |
| fetch failed (asset service unreachable/connection reset)              | 2           |
| Asset service 500 — malformed JSON in LLM response (parse error)       | 1           |
| Asset service 500 — failed to parse <MANIFEST> from final LLM response | 1           |

**Interpretation:** 5/7 failures are availability/timeout issues on the asset-generation service (the longest-running, external-API-dependent stage); 2/7 are LLM output-format robustness issues (non-strict JSON emitted by the model). Planning, code, and building stages recorded **zero** failures.

## 3. Build & Deliverable Metrics (MongoDB builds , n = 33)

| Status                                         | Count |
|------------------------------------------------|-------|
| completed                                      | 13    |
| queued (never started — mirrors orphaned jobs) | 13    |
| failed                                         | 7     |

- Export target: **100 % web** (Godot 4.x HTML5 export).
- buildDuration (measured from job start to build completion, i.e., full pipeline): mean **275.8 s**, min 33.4 s, max 621.5 s over the 20 executed builds — consistent with §2.3.

### 3.1 Deliverable bundle composition (13 completed builds, 78 artifacts)

| Artifact   | Count | Avg size                | Total    | Notes                                        |
|------------|-------|-------------------------|----------|----------------------------------------------|
| index.wasm | 13    | 35.38 MB                | 459.9 MB | Godot 4 web runtime (identical every build)  |
| index.pck  | 13    | 31.91 MB (max 44.75 MB) | 414.9 MB | Game data — the only truly variable artifact |

| Artifact                      | Count | Avg size | Total   | Notes              |
|-------------------------------|-------|----------|---------|--------------------|
| index.js                      | 13    | 331 KB   | 4.3 MB  | Godot JS bootstrap |
| index.png +<br>index.icon.png | 26    | 13.6 KB  | 0.35 MB | Splash + icon      |
| index.html                    | 13    | 4.85 KB  | 0.06 MB | Loader page        |

- **Average playable bundle: 67.7 MB** (min 35.8 MB, max 80.5 MB), 6 files per build.
- Each build is exposed via `gameUrl` ( `/api/games/play/<gameId>/index.html` ) and `downloadUrl` .

## 4. Content Generation Metrics (MongoDB)

### 4.1 AI-generated assets ( `assets` , `n = 154` — all status generated )

| Asset type / entity      | Count |
|--------------------------|-------|
| Audio (SFX + music)      | 35    |
| Sprite — enemy           | 29    |
| Tileset — obstacle       | 27    |
| Sprite — player          | 14    |
| Background — environment | 14    |
| Sprite — collectible     | 13    |
| Sprite — goal            | 13    |
| Sprite — hazard          | 9     |

MIME distribution:  $119 \times \text{image/png}$  ,  $18 \times \text{audio/mpeg}$  ,  $17 \times \text{audio/wav}$  . Player sprites carry animation metadata (avg 6 animations  $\times$  16 frames @ 12 fps, e.g. `idle/run/jump/double-jump/fall/die`).

### 4.2 Conversational planning ( `chats` , `n = 36` )

- 280 total messages, **7.8 messages per chat on average** (max 12) — the requirement-gathering dialogue converges in under 8 turns.

### 4.3 Game plans ( gameplans , n = 29)

- Structured plan documents (archetype, style, theme, parameters, entities, audio, levels): avg 4.3 KB, range 2.7–7.6 KB.

### 4.4 Projects & users ( gameprojects n = 36, users n = 3)

| Project status  | Count |
|-----------------|-------|
| completed       | 13    |
| running (stale) | 13    |
| failed          | 7     |
| draft           | 3     |

- 3 users, average 12 projects/user (max 24 — primary test account).

### 4.5 Usage timeline (pipeline runs per day)

| Date       | Runs |
|------------|------|
| 2026-06-20 | 8    |
| 2026-06-25 | 14   |
| 2026-06-29 | 9    |
| 2026-07-01 | 2    |

## 5. MongoDB Server & Database Metrics

**Database gameforge** : 7 collections, 324 documents, 22 indexes. Data size 797 KB, storage 590 KB, index size 766 KB, total on disk **1.36 MB** — metadata footprint is negligible; all heavy payloads live in object storage.

| Collection     | Docs | Data size | Avg doc | Indexes |
|----------------|------|-----------|---------|---------|
| assets         | 154  | 67.7 KB   | 439 B   | 6       |
| chats          | 36   | 287.2 KB  | 8.0 KB  | 2       |
| gameprojects   | 36   | 22.5 KB   | 625 B   | 4       |
| generationjobs | 33   | 49.3 KB   | 1.5 KB  | 3       |

| Collection | Docs | Data size | Avg doc | Indexes |
|------------|------|-----------|---------|---------|
| builds     | 33   | 169.2 KB  | 5.1 KB  | 3       |
| gameplans  | 29   | 125.9 KB  | 4.3 KB  | 2       |
| users      | 3    | 75.0 KB   | 25 KB   | 2       |

**Server (session snapshot, uptime 7 min):** MongoDB 8.2.11 · 16 open connections (4 active, 0 rejected) · resident memory 214 MB · WiredTiger cache in use 1.15 MB · mean read-op latency **≈ 1.16 ms**, mean command latency ≈ 0.08 ms · 0 write errors.

## 6. MinIO Object Storage Metrics

**Server:** MinIO RELEASE 2025-09-07, single node/single drive, healthy (1/1 drives online), 2.7 % of 956 GiB drive used.

**Bucket** `game-forge-games` : **1,874 objects, 2.20 GiB**, spanning 55 game prefixes.

### 6.1 Storage by content type

| Type  | Objects | Total    | Avg per object | Role                                      |
|-------|---------|----------|----------------|-------------------------------------------|
| .wasm | 27      | 910.9 MB | 34.5 MB        | Godot web runtime (per build)             |
| .png  | 458     | 840.0 MB | 1.88 MB        | AI-generated sprites/tilesets/backgrounds |
| .pck  | 27      | 452.1 MB | 17.1 MB        | Godot game data packs                     |
| .wav  | 110     | 24.8 MB  | 230 KB         | Generated SFX                             |
| .mp3  | 25      | 11.5 MB  | 470 KB         | Generated music                           |
| .js   | 27      | 8.5 MB   | 324 KB         | Web bootstrap                             |

| Type                                            | Objects | Total  | Avg per object | Role                      |
|-------------------------------------------------|---------|--------|----------------|---------------------------|
| .gd                                             | 831     | 2.4 MB | 3 KB           | Generated GDScript source |
| .import / .godot / .cfg / .tscn / .json / .html | 344     | 0.5 MB | —              | Godot project scaffolding |

## 6.2 Storage by area

| Area                                  | Objects | Size            |
|---------------------------------------|---------|-----------------|
| Build outputs (bucket root, per-game) | 142     | 1,372 MB (61 %) |
| godot-project/assets (images)         | 518     | 840 MB (37 %)   |
| godot-project/audio                   | 197     | 36 MB           |
| godot-project/scripts (GDScript)      | 831     | 2.4 MB          |
| Project scaffolding                   | 186     | 0.3 MB          |

## 6.3 Per-game footprint (55 game prefixes)

- Average **40.9 MB per game** (median 34.2 MB, max 117.7 MB), average 34 objects per game.
- 27 retained web-build bundles (some games rebuilt or retained across DB resets — storage holds more builds than the current DB records).

**Optimization finding:** the 35.38 MB Godot runtime ( `index.wasm` + `index.js` ) is byte-identical across builds yet stored per game —  $\approx 875$  MB ( $\approx 40$  % of the bucket) is duplicated runtime. Serving a shared runtime would cut storage by  $\sim 40$  % and per-game footprint from  $\sim 41$  MB to  $\sim 6$  MB of unique data.

# 7. Container Resource Utilization (docker stats snapshot, idle steady-state)

| Container        | CPU %  | Memory    | Net I/O (rx/tx)  | Block I/O (r/w)  |
|------------------|--------|-----------|------------------|------------------|
| game-forge-mongo | 0.44 % | 209.9 MiB | 43.8 kB / 342 kB | 151 MB / 1.6 MB  |
| game-forge-minio | 0.03 % | 137.5 MiB | 10.2 MB / 1.6 GB | 2.51 GB / 270 kB |

| Container           | CPU %  | Memory    | Net I/O (rx/tx)   | Block I/O (r/w)  |
|---------------------|--------|-----------|-------------------|------------------|
| game-forge-backend  | 0.11 % | 63.7 MiB  | 308 kB / 34.6 kB  | 73.7 MB / 0      |
| game-forge-frontend | 0.08 % | 100.5 MiB | 102 kB / 161 kB   | 94.2 MB / 4.1 kB |
| game-forge-planner  | 0.31 % | 49.5 MiB  | 26.6 kB / 10 kB   | 80.6 MB / 0      |
| game-forge-asset    | 0.10 % | 78.3 MiB  | 27.1 kB / 12.6 kB | 22.2 MB / 0      |
| game-forge-code     | 0.00 % | 50.0 MiB  | 3.6 kB / 1.9 kB   | 16 MB / 0        |
| game-forge-builder  | 0.00 % | 51.2 MiB  | 3.4 kB / 1.9 kB   | 13.5 MB / 0      |
| game-forge-server   | 0.00 % | 26.5 MiB  | 2.0 kB / 126 B    | 3.7 MB / 0       |
| assets-mcp          | 0.13 % | 74.1 MiB  | 1.9 kB / 126 B    | 52 MB / 0        |

Total idle memory footprint of the full 10-container stack: **≈ 841 MiB** (5.4 % of the 15.33 GiB host limit). MinIO's 1.6 GB transmitted / 2.51 GB block-read reflects serving game bundles and asset uploads.

## 8. Key Findings & Recommendations

- 1. **Throughput:** a full prompt-to-playable-game pipeline completes in **~4.6 min on average** (best case < 2 min), with asset generation consuming ~79 % of that time — the clear target for parallelization or caching.
- 2. **Reliability:** 65 % terminal success rate. All failures trace to the assets stage: timeouts/unreachability (5) and non-strict LLM JSON output (2). Retry-with-backoff on asset calls and schema-constrained/repair-parsing of LLM responses would address 100 % of observed failures.
- 3. **Orphaned state:** 13 jobs/builds/projects stranded in `processing` / `queued` / `running` after service restarts — a startup reconciliation sweep (mark stale `processing` jobs failed) would keep statistics and UX clean.
- 4. **Storage efficiency:** deduplicating the identical Godot web runtime across builds would reclaim ~40 % of object storage.
- 5. **Scalability headroom:** metadata DB is 1.36 MB for 36 projects (~38 KB/project) and the idle stack uses < 6 % of host RAM; storage grows ~41 MB/game (~6 MB/game if the runtime is shared) — the architecture comfortably scales to hundreds of projects on current hardware.

Raw extraction data: [raw\\_metrics.json](#) · per-run timings: [pipeline\\_runs.csv](#). Extraction method: `mongosh aggregations inside game-forge-mongo ; mc du / mc ls --recursive / mc admin info inside game-forge-minio ; docker stats on the host`.